```
"""
================================================================================
JAGAPRIYAN B
CH.SC.U4CSE24216
LAB EXERCISE 7: DECISION TREE CLASSIFIERS

Topics Covered:
  Exercise 1: House Buying Prediction based on Age and Income (Splitting Criteria & Purity)
  Exercise 2: Weather-based Activity Classification ("Go Out" vs "Stay In")
================================================================================
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sn
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn import metrics
from sklearn.preprocessing import LabelEncoder

print("="*70)
print("EXERCISE 7.1: HOUSE BUYING DECISION TREE (AGE & INCOME SPLIT)")
print("="*70)
# Scenario from Lab Manual (Page 30):
# Two variables: age and income determining whether someone buys a house.
# 70% of people over age 30 bought a house -> age split at 30 -> second node: income.

np.random.seed(42)
n_samples = 200

ages = np.random.randint(20, 65, n_samples)
incomes = np.random.randint(25000, 120000, n_samples)
buys_house = []

for a, inc in zip(ages, incomes):
    # Rule reflecting the lab scenario:
    if a > 30:
        if inc > 55000:
            buys_house.append(1 if np.random.rand() < 0.85 else 0)
        else:
            buys_house.append(1 if np.random.rand() < 0.40 else 0)
    else:
        if inc > 75000:
            buys_house.append(1 if np.random.rand() < 0.35 else 0)
        else:
            buys_house.append(0 if np.random.rand() < 0.90 else 1)

df_house = pd.DataFrame({
    'Age': ages,
    'Income': incomes,
    'BuysHouse': buys_house
})

print("Dataset sample:")
print(df_house.head())

X_house = df_house[['Age', 'Income']]
y_house = df_house['BuysHouse']

X_train_h, X_test_h, y_train_h, y_test_h = train_test_split(
    X_house, y_house, test_size=0.2, random_state=42
)

# Decision Tree with Entropy/Gini (as in the manual, max_depth=3 for clear interpretable nodes)
dt_house = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
dt_house.fit(X_train_h, y_train_h)

y_pred_h = dt_house.predict(X_test_h)

print(f"\nAccuracy Score: {metrics.accuracy_score(y_test_h, y_pred_h):.4f}")
print("Confusion Matrix:\n", metrics.confusion_matrix(y_test_h, y_pred_h))

# Display Decision Tree Rules in Text
print("\n--- Decision Tree Rules ---")
print(export_text(dt_house, feature_names=['Age', 'Income']))

# Visualizing the Decision Tree
plt.figure(figsize=(12, 6))
plot_tree(dt_house, feature_names=['Age', 'Income'], class_names=['No Buy', 'Buys House'],
          filled=True, rounded=True, fontsize=10)
```

```
          filled=True, rounded=True, fontsize=10)
plt.title("Decision Tree for House Buying Prediction")
plt.show()


print("\n" + "="*70)
print("EXERCISE 7.2: WEATHER-BASED ACTIVITY: 'GO OUT' OR 'STAY IN'")
print("="*70)
# Lab Manual Scenario:
# Weather conditions: Sunny, Rainy, Overcast
# Temperature: Hot, Mild, Cool
# Humidity: High, Normal
# Wind: Weak, Strong
# Decision: "Go Out" vs "Stay In"

weather_data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rainy', 'Rainy', 'Rainy', 'Overcast',
                'Sunny', 'Sunny', 'Rainy', 'Sunny', 'Overcast', 'Overcast', 'Rainy'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool',
                    'Mild', 'Cool', 'Mild', 'Mild', 'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal',
                 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong',
             'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
    'Action': ['Stay In', 'Stay In', 'Go Out', 'Stay In', 'Go Out', 'Stay In', 'Go Out',
               'Stay In', 'Go Out', 'Go Out', 'Go Out', 'Go Out', 'Go Out', 'Stay In']
}

df_weather = pd.DataFrame(weather_data)
print("Weather Activity Dataset:")
print(df_weather)

# Label Encoding for categorical features
le_dict = {}
df_weather_encoded = df_weather.copy()
for col in ['Outlook', 'Temperature', 'Humidity', 'Wind', 'Action']:
    le = LabelEncoder()
    df_weather_encoded[col] = le.fit_transform(df_weather[col])
    le_dict[col] = le
    print(f"Mapping for {col}: {dict(zip(le.classes_, le.transform(le.classes_)))}")

X_weather = df_weather_encoded[['Outlook', 'Temperature', 'Humidity', 'Wind']]
y_weather = df_weather_encoded['Action']

# Fit Decision Tree Classifier
dt_weather = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt_weather.fit(X_weather, y_weather)

# Predict for a new weekend scenario: Sunny, Mild temperature, Normal humidity, Weak wind
sunny_enc = le_dict['Outlook'].transform(['Sunny'])[0]
mild_enc = le_dict['Temperature'].transform(['Mild'])[0]
norm_enc = le_dict['Humidity'].transform(['Normal'])[0]
weak_enc = le_dict['Wind'].transform(['Weak'])[0]

sample_scenario = pd.DataFrame([[sunny_enc, mild_enc, norm_enc, weak_enc]],
                               columns=['Outlook', 'Temperature', 'Humidity', 'Wind'])
predicted_action_code = dt_weather.predict(sample_scenario)[0]
predicted_action_label = le_dict['Action'].inverse_transform([predicted_action_code])[0]

print(f"\nScenario: Sunny, Mild Temp, Normal Humidity, Weak Wind")
print(f"Predicted Action: -> {predicted_action_label}")

# Visualize the Tree
plt.figure(figsize=(14, 7))
plot_tree(dt_weather,
          feature_names=['Outlook', 'Temperature', 'Humidity', 'Wind'],
          class_names=le_dict['Action'].classes_,
          filled=True, rounded=True, fontsize=10)
plt.title("Decision Tree: 'Go Out' vs 'Stay In' Based on Weather")
plt.show()
```